LETTER

Formal and Empirical Analysis of an Authorization Decision Inconsistency in SAGA

Anchen GAN[†a)] ***and*** **Libin WANG**[†], ***Nonmembers***

SUMMARY Authentication does not guarantee compliance with a receiver's contact policy. We identify an order-dependent SAGA matcher defect converting specified denial into permission. Controlled Provider/receiver executions confirm propagation to token issuance and storage. In an evidence-linked symbolic scenario, ProVerif reports conditional `ChatAccept` reachability while the prior-`TokenIssue` correspondence holds, separating token origin from policy compliance.

key words: *authorization, access control, formal verification, AI agents, ProVerif*

1. Introduction

LLM-based agents increasingly use external tools and services [1], [2]. Authentication establishes identity, whereas receiver-side authorization determines whether the identified agent may communicate [3]. Authentication and token secrecy alone do not establish compliance with the receiver's intended contact policy.

SAGA [4] governs communication between registered agents using contact policies and access-control tokens. Its policy matcher implements the receiver's authorization decision. We investigate: *Can a mismatch between the receiver's intended policy and its implementation permit a denied agent, and how far can that permission propagate through the protocol?*

We identify an order-dependent matcher defect that violates SAGA's most-specific-rule semantics, yielding specification denial but implementation permission. An authorization coverage diagnostic identifies a missing proof obligation without discovering the implementation defect. Controlled executions of production Provider and receiver code paths confirm propagation to token issuance and storage. Under the corresponding evidence-linked fixed symbolic divergence, ProVerif reports `ChatAccept` reachable while the prior-`TokenIssue` correspondence holds. Actual application-message acceptance is not established.

Our contributions are (1) identifying and explaining a concrete authorization decision inconsistency in SAGA; (2) analyzing its conditional symbolic consequences using ProVerif; and (3) validating its propagation to token issuance and storage through controlled production-code execution.

(1) Related work.

Prior work examines authorization languages and semantics [5], [6], and policy verification [7]. SPKI [3] and Macaroons [8] provide related authorization abstractions. ProVerif and Tamarin support symbolic protocol analysis [9], [10]. Our analysis connects a concrete policy-implementation inconsistency to its consumer and symbolic consequences.

2. Authorization Decision Inconsistency

(1) Intended authorization semantics.

SAGA's Provider handles contact requests under the receiver's ordered policy, which maps identity patterns to integer budgets. The Provider and receiver independently call the production matcher: the Provider gates one-time key (OTK) release, and the receiver rechecks authorization before token issuance.

SAGA, Section IV-D.1 [4], specifies selection of the most-specific matching rule; `aid_specificity()` scores the patterns. A positive budget permits the request; a negative budget denies it. We consider fixed policies and valid identities, excluding equal-specificity conflicts and zero budgets.

(2) Root cause and observation.

The tested production matcher is `match()` in `saga/common/contact_policy.py`. The artifact binds the archived matcher observation to pinned upstream source provenance.[†] Its `best_pattern` starts at `None`, whose specificity is -1. A more-specific matching rule updates the budget but never `best_pattern`. Later matches still compare against -1, allowing a less-specific wildcard to overwrite the budget. In the recorded case, `aid_specificity()` scores `alice@example.com:*` as 70 and `*` as 0; their budgets are -1 and 10, respectively.

Table 1 Specification and matcher budgets for two rule orders.

Rule order	Spec. budget	Matcher budget
Specific deny, then wildcard allow	-1	10
Wildcard allow, then specific deny	-1	-1

The independent finite specification evaluator selects

[†]The authors are with the School of Computer Science, South China Normal University, Guangzhou 510631, China.

a) E-mail: 1020268262@qq.com

DOI: 10.1587/transfun.E0.A.1

[†]Reproduction artifact: `github.com/g1020268262-a11y/SAGA-Authorization-Analysis-Artifact`.

Copyright © 200x The Institute of Electronics, Information and Communication Engineers

denial for both orders in Table 1. Specific deny followed by wildcard allow changes the specified −1 into an observed 10; reversing the order restores denial. This counterexample establishes an implementation-level authorization decision inconsistency for the recorded policy, with permission dependent on rule order.

3. Formal Authorization Analysis

(1) Authorization properties and ProVerif queries.

Token-origin consistency does not establish compliance with the receiver's intended policy. For a static receiver policy, the desired authorization condition is

$$\mathsf{TokenIssue}(R, I, t) \Longrightarrow \mathsf{Allow}_{\mathrm{spec}}(P_R, I),$$

where R is the receiver and token issuer, I is the initiator, and P_R is the receiver's contact policy. $\mathsf{Allow}_{\mathrm{spec}}$ denotes permission under the intended policy semantics. This condition is neither an executed ProVerif query nor a proved theorem.

The archived ProVerif 2.05 model evaluates two queries under a fixed specification-Deny/implementation-Allow divergence. Query listings use source variables; result lines abbreviate event parameters.

Query 1: Message-acceptance reachability.

```
query receiver: bitstring, sender: bitstring,
      tok: bitstring, msg: bitstring;
  event(ChatAccept(receiver,sender,tok,msg)).

RESULT not event(ChatAccept(...)) is false.
```

The false negated-event result establishes that `ChatAccept` is reachable in the fixed symbolic scenario.

Query 2: Token-origin correspondence.

```
query receiver: bitstring, sender: bitstring,
      tok: bitstring, msg: bitstring;
  event(ChatAccept(receiver,sender,tok,msg))
  ==> event(TokenIssue(receiver,sender,tok)).

RESULT event(ChatAccept(...))
  ==> event(TokenIssue(...)) is true.
```

The verified non-injective correspondence requires a preceding `TokenIssue` for the same receiver, initiator, and encrypted token. Together, the results show that symbolic message acceptance is reachable despite intended-policy denial, while token-origin consistency remains satisfied. The acceptance witness includes token issuance; no standalone `TokenIssue` reachability query was executed.

SAGA's existing authentication and token-secrecy verification does not establish this authorization obligation. An archived coverage diagnostic reports an added authorization correspondence as false because `PolicyAllow` is not linked to policy enforcement. The diagnostic concerns `Accept`, which denotes token receipt and verification rather than `ChatAccept`. It does not identify the Python matcher defect.

(2) Scenario and protocol interpretation.

A bounded bridge compares the archived matcher observation with an independently evaluated policy decision, checks source evidence, and instantiates the verified Deny/Allow decision pair in a fixed ProVerif scenario. This is scenario reconstruction, not Python-to-ProVerif refinement.

The symbolic initiator has a valid registered identity, follows the prescribed peer process, and cannot alter the receiver's fixed policy. The Provider and Certificate Authority are trusted, private keys remain uncompromised, and a private symbolic channel abstracts authenticated and confidential transport.

In the model, `TokenIssue`(R, I, t) occurs after the corresponding `ActiveToken` entry is stored and before the token is sent. `ChatAccept`(R, I, t, m) occurs after message receipt and checks of the sender identity and the stored receiver, initiator, token, and sender-key binding.

ProVerif assumes the checked authorization divergence; it does not execute the Python matcher. The resulting reachability is conditional on the symbolic model and its assumptions, not an arbitrary-attacker or deployed exploit. Controlled implementation validation reaches token issuance and storage, but does not execute application-message acceptance.

4. Consumer-Side Propagation Validation

The validation asks whether incorrect permission survives the production consumer checks and reaches token issuance and storage. The controlled harness drives the Provider `/access` route and receiver handshake, including OTK release and consumption. Archived source hashes bind the matcher, consumers, and token-related cryptographic code to the pinned upstream baseline.

Signed registration material, local certificates, and in-memory transport provide fixtures while production cryptographic checks remain active. Both consumers call the original matcher; wrappers record and return its result unchanged. Fixtures preset neither authorization decisions nor token state.

(1) Control design.

The labels A/A, D/A, and D/D denote the specification and observed matcher decisions, respectively, rather than Provider and receiver outcomes. Both consumers use the same policy within each case. A/A assigns budget 10 to both rules and serves as a permitted-path control. D/A places a specific deny (−1) before a wildcard allow (10), producing the observed authorization inconsistency. D/D reverses this order, restoring denial and testing Provider-side blocking. Each case ran once with fresh state and one OTK; the D/D case did not reach the receiver.

A/A reaches token sending and storage. In D/A, Table 2 confirms that permission from both production matchers reaches token issuance and `active_tokens` storage despite specification denial. One token is generated, sent, and stored. D/D returns HTTP 403 without OTK release or state change, establishing only Provider-side blocking; receiver rejection remains untested.

These observations establish controlled consumer prop-

agation, not application message acceptance. No application message is sent: a boundary stub intercepts `receive_conversation` before the original handler executes.

5. Scope and Conclusion

For the recorded SAGA policy, order-dependent matching authorizes an agent denied by the intended most-specific semantics. The resulting permission reaches token issuance and storage through controlled production consumers; the corresponding fixed symbolic model permits `ChatAccept` while its prior-`TokenIssue` correspondence holds. Authentication and token-origin consistency are insufficient to establish compliance with the receiver's intended authorization policy.

These findings concern the recorded case, tested policies, fixtures, and symbolic assumptions. They establish neither arbitrary-policy correctness nor Python-to-ProVerif refinement or an arbitrary-attacker or deployed exploit. Consumer execution stops at token issuance and storage; message acceptance belongs only to the symbolic model.

Table 2 Production consumer results for the divergent D/A case.

Evidence point	Divergent case result
Specification	Deny (−1)
Provider matcher	Allow (10)
Provider `/access`	HTTP 200; OTK released
Receiver matcher	Allow (10)
Receiver	OTK consumed
Endpoint	Token issued and stored

References

[1] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W.X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," Frontiers of Computer Science, vol.18, no.6, p.186345, 2024.

[2] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," Advances in Neural Information Processing Systems 37 (NeurIPS 2024), pp.82895–82920, 2024.

[3] C. Ellison, B. Frantz, B. Lampson, R. Rivest, B. Thomas, and T. Ylonen, "SPKI certificate theory," RFC 2693, RFC Editor, Sept. 1999. Experimental.

[4] G. Syros, A. Suri, J. Ginesin, C. Nita-Rotaru, and A. Oprea, "SAGA: A security architecture for governing AI agentic systems," Network and Distributed System Security (NDSS) Symposium, Internet Society, Feb. 2026.

[5] M.Y. Becker and P. Sewell, "Cassandra: Distributed access control policies with tunable expressiveness," 5th IEEE International Workshop on Policies for Distributed Systems and Networks (POLICY), 2004.

[6] M.Y. Becker, C. Fournet, and A.D. Gordon, "Design and semantics of a decentralized authorization language," 20th IEEE Computer Security Foundations Symposium (CSF), pp.3–15, IEEE, 2007.

[7] K. Fisler, S. Krishnamurthi, L.A. Meyerovich, and M.C. Tschantz, "Verification and change-impact analysis of access-control policies," 27th International Conference on Software Engineering (ICSE), pp.196–205, 2005.

[8] A. Birgisson, J.G. Politz, Ú. Erlingsson, A. Taly, M. Vrable, and M. Lentczner, "Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud," Network and Distributed System Security Symposium (NDSS), Internet Society, 2014.

[9] B. Blanchet, "Modeling and verifying security protocols with the applied pi calculus and ProVerif," Foundations and Trends in Privacy and Security, vol.1, no.1–2, pp.1–135, 2016.

[10] S. Meier, B. Schmidt, C. Cremers, and D. Basin, "The TAMARIN prover for the symbolic analysis of security protocols," Computer Aided Verification (CAV), Lecture Notes in Computer Science, vol.8044, pp.696–701, Springer, 2013.